CS 550 Final Project Proposal: Replication of the Hawkeye Cache Replacement Policy

Vito Wang, Endong Dai, and Xiaotian Tong

***Abstract*—This project replicates the Hawkeye cache replacement policy [1] in gem5 and evaluates it on SPEC CPU2017. Hawkeye learns from Belady's optimal replacement algorithm. It reconstructs the decisions that the optimal policy would have made on past cache accesses and uses them to train a predictor that decides which cache lines to keep. We will validate each component with unit tests and logs, check the reconstructed decisions against an offline simulation of the optimal policy, and compare Hawkeye's miss rate and performance with the replacement policies that gem5 already provides, such as LRU, DRRIP, and SHiP.**

I. Introduction & Problem Statement

The last-level cache (LLC) is the last on-chip structure that can hide the long latency of DRAM, so every LLC miss stalls the processor for hundreds of cycles. When the LLC is full, the replacement policy chooses which line to evict, and this choice largely decides the miss rate. The common least-recently-used (LRU) policy works well when recently used data is soon reused. However, it fails when a program cycles through a working set larger than the cache, because every line is then evicted just before it is reused, and when streaming data that is never reused pushes useful lines out. Improved policies such as RRIP [2] and SHiP [3] handle some of these patterns, but each is built around a particular class of access patterns.

Belady's algorithm [4], also called OPT, is the optimal policy. It evicts the line whose next use lies furthest in the future. OPT cannot be built in hardware because it needs future knowledge. Hawkeye [1] avoids this problem by looking at the past instead. It computes what OPT would have done on past accesses, learns which load instructions tend to bring in lines that OPT would keep, and uses this to guide future evictions. The original paper evaluated Hawkeye in the Cache Replacement Championship simulator on SPEC CPU2006. gem5 does not include Hawkeye, so it has not been evaluated with gem5's out-of-order CPU model or with SPEC CPU2017.

This project aims to implement Hawkeye [1] in gem5 [5], [6]. The goal is to verify its functional correctness and to compare its miss rate and performance with the baseline policies in gem5.

II. Related Work

Classic replacement policies rank lines by recency or frequency, e.g., LRU and LFU [7]. RRIP [2] instead predicts how soon each line will be re-referenced, using a 2- or 3-bit counter per line, and inserts new lines with a long predicted interval so that data that is never reused leaves the cache quickly. DRRIP chooses between two RRIP variants at run time through set dueling [8], which compares the policies on a few dedicated sets. A previous CS 550 project evaluated DRRIP, SRRIP, and BRRIP against LRU in gem5. We use these policies as baselines.

Other policies predict reuse for each load instruction. SHiP [3] learns, for each program counter (PC), whether the lines it inserts are reused, and SDBP [9] predicts dead blocks from a small sampler. Both learn from the behavior of a heuristic policy, so they inherit its limits.

Hawkeye [1] instead learns from OPT itself. Its authors won the 2nd Cache Replacement Championship (CRC2) [10], and we will use the CRC2 code as a reference. Its follow-up, Mockingjay [11], predicts reuse distances instead of a binary decision. We leave Mockingjay out of scope.

III. Research Questions

- **What is the Hawkeye policy, and can its reconstruction of OPT be implemented correctly in gem5?**
- **How do Hawkeye's LLC miss rate and IPC compare with LRU, DRRIP, and SHiP, and how close does Hawkeye come to OPT?**
- **How sensitive is Hawkeye to its hardware budget and to the LLC size?**

IV. Proposed Solution

Jain and Lin proposed Hawkeye [1], which has three parts.

OPTgen computes whether each past access would have hit under OPT. For a line X, the time from one access to X to the next access to X is X's usage interval. OPT could have kept X in the cache during this interval only if fewer lines than the associativity were already being kept at every point in it. OPTgen therefore keeps an *occupancy vector* per set, which counts how many lines OPT holds at each point in time. When X is accessed again, OPTgen checks the vector over X's usage interval. If every entry is below the associativity, OPT would have hit, and OPTgen increments those entries. Otherwise, OPT would have missed. The vector covers a history of $8\times$ the associativity, and each entry represents 4 accesses, so a 16-way set needs 32 entries of 4 bits.

Sampling. OPTgen runs on only 64 randomly chosen sets. A separate Sampled Cache records the recent accesses to these sets. Each entry holds a partial tag, the PC of the load, and a

1-byte timestamp. 2400 entries, replaced in LRU order, give the 8× history.

The Hawkeye Predictor learns from OPTgen. It is a table of 8K 3-bit counters indexed by a 13-bit hash of the PC. When OPTgen decides that the previous access to X would have hit under OPT, the counter of the PC that made that access is incremented. Otherwise, it is decremented. On every LLC access, the counter of the current PC predicts whether the line is *cache-friendly* (high bit set) or *cache-averse*.

Replacement uses a 3-bit RRIP value per line. A cache-averse line gets the value 7, which marks it for eviction first. A cache-friendly line gets 0, and when it is inserted, the other lines in the set age by one, up to a maximum of 6. The victim is any line with value 7. If no such line exists, Hawkeye evicts the line with the highest value, i.e., the oldest cache-friendly line, and decrements the counter of its PC if the line is in the Sampled Cache. The paper reports a total budget of about 16 KB for a 2 MB LLC.

V. Evaluation

A. Methodology

We will use gem5 v23.0.0.1 on the course AWS instance, in syscall-emulation (SE) mode with the x86 out-of-order CPU model (O3CPU). Following the original paper, the cache hierarchy has 32 KB 4-way L1 instruction and data caches, a 256 KB 8-way L2, and a 2 MB 16-way LLC. The course configuration has only two cache levels, so we will add the LLC. We will add the following components to gem5:

- **Hawkeye replacement policy**: a new class for gem5's replacement-policy interface, which passes the requesting packet, and thus the PC, on every insertion and hit, as the existing SHiP policy uses.
- **Per-line state**: the RRIP value, the PC signature, and the prediction, stored with each line because the eviction call does not see the packet.
- **OPTgen and the Sampled Cache**: occupancy vectors and access history for the 64 sampled sets.
- **Hawkeye Predictor**: the 8K-entry table of 3-bit counters.
- **Requests without a PC**: prefetches and writebacks, which need a fixed signature.
- **Logs and statistics**: a debug flag that prints each decision.

B. Functional Validation

The functional correctness of the Hawkeye policy will be tested in three ways.

a) Unit testing with logs: We will write a Python simulator of OPT and feed it the same address trace as OPTgen. For each access, the log will show OPTgen's hit or miss decision, the predictor counter, and the RRIP values, which we will compare with the Python results and, for short traces, with values calculated by hand, such as the example in the original paper.

b) Microbenchmarks: We will write small C programs with known behavior. A loop over an array slightly larger than the LLC makes LRU miss on every access, whereas OPT keeps part of the array. A reused array mixed with a stream tests whether streaming lines are evicted first. An array that fits in the LLC should hit under every policy.

c) Execution correctness: The replacement policy must not change program results. The outputs of all test programs will be compared with those under LRU.

C. Performance Impact

We will compare Hawkeye with LRU, BRRIP, DRRIP, and SHiP, all of which gem5 already provides, and with OPT from the offline simulator as an upper bound. The workloads are the SPEC CPU2017 [12] speed benchmarks that the course scripts support, such as mcf, omnetpp, xalancbmk, and gcc. For each benchmark, we will skip the initialization, warm up the caches, and then measure a fixed number of instructions. The metrics are LLC misses per kilo-instruction (MPKI), the miss reduction over LRU, and IPC. We will also vary the number of sampled sets, the predictor size, and the LLC size. We will estimate the hardware cost from the storage of the added structures, and we will evaluate a shared LLC on multiple cores if time permits.

VI. Milestones

The project is broken down into five milestones that span 5 weeks of work.

- **Milestone 1: Setup and Initial Research (Week 1):** We will add the LLC and the O3CPU to the gem5 configuration, run the baseline policies, study the Hawkeye paper and the CRC2 code, and confirm that the PC reaches the LLC.
- **Milestone 2: Hawkeye Implementation (Week 2):** We will implement OPTgen, the Sampled Cache, the predictor, and the replacement policy, together with the Python OPT simulator.
- **Milestone 3: Functional Validation (Week 3):** We will run the unit tests and microbenchmarks and fix any differences from the expected behavior.
- **Milestone 4: Performance Evaluation (Week 4):** We will run SPEC CPU2017 with every policy and perform the sensitivity studies.
- **Milestone 5: Final Evaluation and Reporting (Week 5):** We will analyze the results and write the final report and presentation.

VII. Division of Labor

Vito will lead the implementation of OPTgen, the predictor, and the replacement policy in gem5. Endong will build the Python OPT simulator and the microbenchmarks and lead the functional validation. Xiaotian will set up the gem5 configuration and the SPEC CPU2017 runs and lead the performance evaluation. All three will analyze the results and write the final report.

REFERENCES

[1] A. Jain and C. Lin, "Back to the future: Leveraging Belady's algorithm for improved cache replacement," in *Proc. ISCA*, 2016.

[2] A. Jaleel, K. B. Theobald, S. C. Steely Jr., and J. Emer, "High performance cache replacement using re-reference interval prediction (RRIP)," in *Proc. ISCA*, 2010.

[3] C.-J. Wu, A. Jaleel, W. Hasenplaugh, M. Martonosi, S. C. Steely Jr., and J. Emer, "SHiP: Signature-based hit predictor for high performance caching," in *Proc. MICRO*, 2011.

[4] L. A. Belady, "A study of replacement algorithms for a virtual-storage computer," *IBM Systems Journal*, vol. 5, no. 2, pp. 78–101, 1966.

[5] N. Binkert *et al.*, "The gem5 simulator," *ACM SIGARCH Comput. Archit. News*, vol. 39, no. 2, pp. 1–7, 2011.

[6] J. Lowe-Power *et al.*, "The gem5 simulator: Version 20.0+," arXiv:2007.03152, 2020.

[7] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2019.

[8] M. K. Qureshi, A. Jaleel, Y. N. Patt, S. C. Steely Jr., and J. Emer, "Adaptive insertion policies for high performance caching," in *Proc. ISCA*, 2007.

[9] S. M. Khan, Y. Tian, and D. A. Jiménez, "Sampling dead block prediction for last-level caches," in *Proc. MICRO*, 2010.

[10] The 2nd Cache Replacement Championship (CRC2), held with ISCA 2017.

[11] I. Shah, A. Jain, and C. Lin, "Effective mimicry of Belady's MIN policy," in *Proc. HPCA*, 2022.

[12] Standard Performance Evaluation Corporation, "SPEC CPU2017," https://www.spec.org/cpu2017/.